

TRƯỜNG: ĐH CNTP TP.HCM KHOA: CÔNG NGHỆ THÔNG TIN BỘ MÔN: KHD&TTNT MH: TH TRÍ TUỆ NHÂN TẠO	BUỔI 2. GIẢI THUẬT TÌM KIẾM A^{KT} và A^*	
--	---	---

A. MỤC TIÊU

- Cài đặt thuật toán tìm kiếm AKT và A^* .
- Các ứng dụng.

B. DỤNG CỤ - THIẾT BỊ THÍ NGHIỆM CHO MỘT SV

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

I. Tóm tắt lý thuyết

Thuật giải A^{KT}

Bước 1:

Mọi đỉnh, cũng như các hàm g , h , f chưa biết;

Mở đỉnh đầu tiên S , gán $g(S) = 0$;

Sử dụng tri thức bổ sung để ước tính hàm $h(S)$;

Tính $f(S) = g(S) + h(S)$;

Bước 2: Chọn đỉnh mở có f là nhỏ nhất và gọi là đỉnh N

+ Nếu N là đích: đường đi từ đỉnh ban đầu đến đỉnh N là ngắn nhất và bằng $g(N)$.

Dừng (*Thành công*).

+ Nếu không tồn tại đỉnh mở nào: cây biểu diễn vấn đề không tồn tại đường đi tới mục tiêu. Dừng (*Thất bại*).

+ Nếu có 2 đỉnh mở trở lên có cùng giá trị f nhỏ nhất: kiểm tra xem những đỉnh đó có đỉnh nào là đích hay không.

Nếu có: đường đi từ đỉnh ban đầu đến đỉnh N là ngắn nhất và bằng $g(N)$. Dừng (*Thành công*).

Nếu không có: chọn ngẫu nhiên một trong các đỉnh đó và gọi đỉnh đó là N .

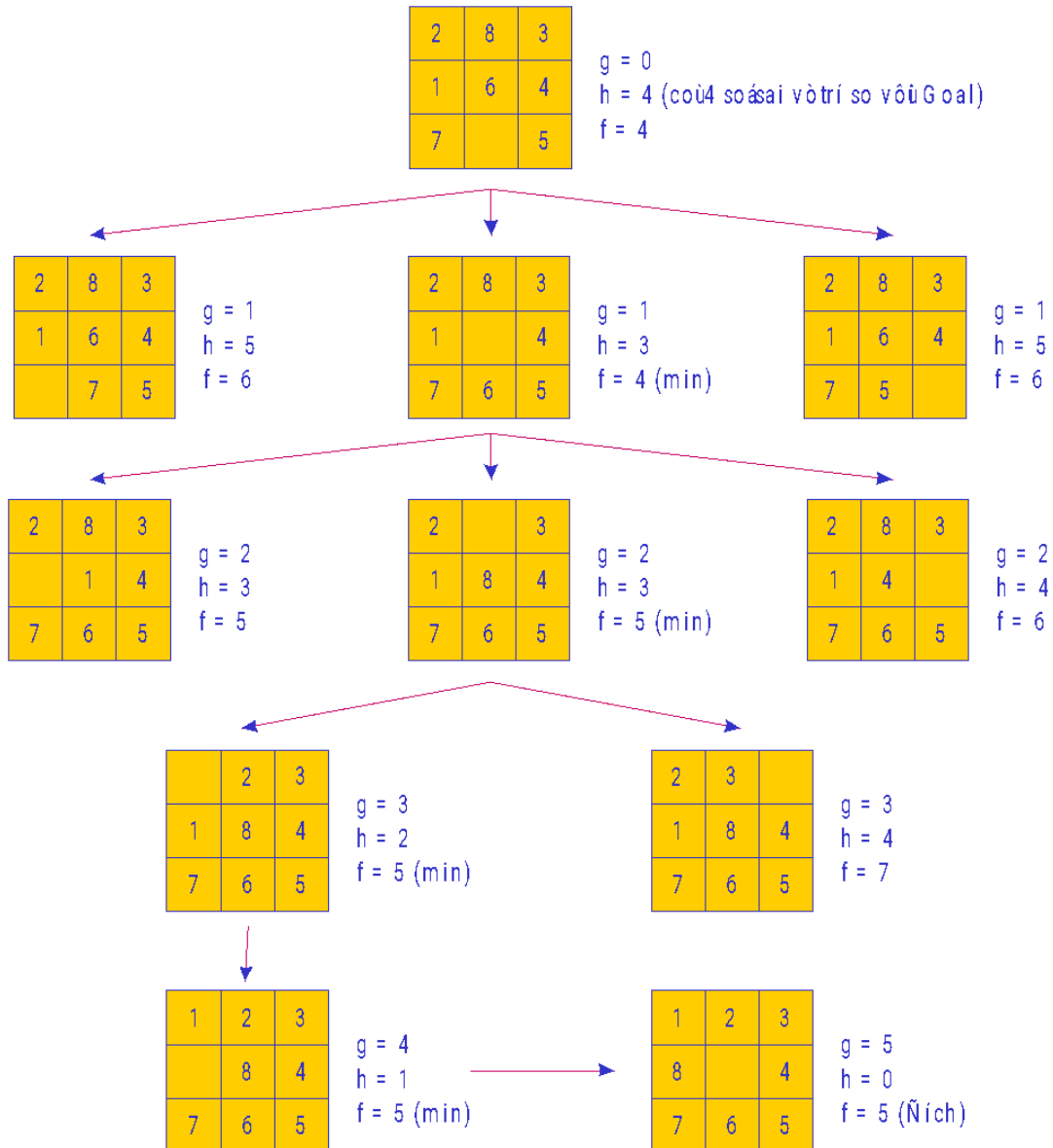
Bước 3: Đóng đỉnh N , mở mọi đỉnh sau N . Với mỗi đỉnh S sau N , tính:

$$g(S) = g(N) + \text{cost}(S \rightarrow N);$$

Sử dụng tri thức bổ sung để tính $h(S)$ và $f(S)$: $f(S) = g(S) + h(S)$

Bước 4: Quay lại bước 2.

Ví dụ:



Thuật toán tìm kiếm A * trong Python hay nói chung về cơ bản là một bài toán trí tuệ nhân tạo được sử dụng để tìm đường (từ điểm A đến điểm B) và truyền qua Biểu đồ. Thuật toán này linh hoạt và có thể được sử dụng trong nhiều ngữ cảnh. Thuật toán tìm kiếm A * sử dụng chi phí đường dẫn theo kinh nghiệm, chi phí của điểm bắt đầu và điểm kết thúc. Thuật toán này được Peter Hart, Nils Nilsson và Bertram Raphael công bố lần đầu tiên vào năm 1968.

Thuật toán này là dạng nâng cao của thuật toán BFS (Breadth-first search), tìm kiếm đường đi ngắn hơn so với đường đi dài. Nó là một giải pháp hoàn chỉnh cũng như tối ưu để giải quyết các vấn đề về đường đi và lưới (grid).

Tối ưu ở đây có nghĩa là tìm chi phí ít nhất từ điểm bắt đầu đến điểm kết thúc.

Complete - Có nghĩa là nó sẽ tìm thấy tất cả các đường đi có sẵn từ điểm bắt đầu đến điểm cuối.

Một số khái niệm cơ sở của A^* :

- $g(n)$: chi phí đường đi thực tế từ điểm bắt đầu đến điểm hiện tại.
- $h(n)$: chi phí đường đi thực tế từ điểm hiện tại đến điểm kết thúc.
- $f(n)$: chi phí đường đi thực tế từ điểm bắt đầu đến điểm kết thúc.

Để thực hiện thuật toán A^* , chúng ta phải sử dụng hai mảng là OPEN và CLOSE. Trong đó mảng OPEN chứa các điểm chưa được kiểm tra và mảng CLOSE chứa các điểm đã được kiểm tra.

Thuật toán:

Bước 1: đặt điểm bắt đầu vào mảng OPEN và tính giá trị $f(n)$ của nó.

Bước 2: lấy ra một điểm từ mảng OPEN mà có giá trị $f(n)$ tương ứng là nhỏ nhất gọi là remove node. Nếu điểm lấy ra là điểm kết thúc thì báo thành công và dừng thuật toán, ngược lại chuyển sang bước 3.

Bước 3: tìm tất cả các điểm kề cận và tính giá trị $f(n)$ của chúng, đặt tất cả các điểm kề cận tìm được vào mảng OPEN, và đặt remove node vào mảng CLOSE.

Bước 4: quay lại Bước 2.

Bước 5: Thoát.

II. Bài tập hướng dẫn mẫu

a. Thuật giải AKT vào bài toán 8 puzzle($n=3$)

```
#import một số thư viện hỗ trợ
import copy
# library for the Priority Queue
from heapq import heappush, heappop

# Giả sử puzzle(n=3)
n = 3

# 4 vị trí dịch chuyển tương ứng bottom, left, top, right
rows = [ 1, 0, -1, 0 ]
cols = [ 0, -1, 0, 1 ]

# Tạo một lớp hàng đợi creating a class for the Priority Queue
class priorityQueue:
    def __init__(self):
        self.heap = []
```

```

# Inserting a new key 'key'
def push(self, key):
    heappush(self.heap, key)

# funct to remove the element that is minimum,
# from the Priority Queue
def pop(self):
    return heappop(self.heap)

# funct to check if the Queue is empty or not
def empty(self):
    if not self.heap:
        return True
    else:
        return False

# structure of the node
class nodes:

    def __init__(self, parent, mats, empty_tile_posi,
                  costs, levels):

        # This will store the parent node to the
        # current node And helps in tracing the
        # path when the solution is visible
        self.parent = parent

        # Useful for Storing the matrix
        self.mats = mats

        # useful for Storing the position where the
        # empty space tile is already existing in the matrix
        self.empty_tile_posi = empty_tile_posi

        # Store no. of misplaced tiles
        self.costs = costs

        # Store no. of moves so far
        self.levels = levels

        # This func is used in order to form the
        # priority queue based on
        # the costs var of objects
        def __lt__(self, nxt):
            return self.costs < nxt.costs

# method to calc. the no. of
# misplaced tiles, that is the no. of non-blank
# tiles not in their final posi
def calculateCosts(mats, final) -> int:

    count = 0
    for i in range(n):
        for j in range(n):
            if ((mats[i][j]) and
                (mats[i][j] != final[i][j])):
                count += 1

    return count

def newNodes(mats, empty_tile_posi, new_empty_tile_posi,

```

```

        levels, parent, final) -> nodes:

# Copying data from the parent matrixes to the present matrixes
new_mats = copy.deepcopy(mats)

# Moving the tile by 1 position
x1 = empty_tile_posi[0]
y1 = empty_tile_posi[1]
x2 = new_empty_tile_posi[0]
y2 = new_empty_tile_posi[1]
new_mats[x1][y1], new_mats[x2][y2] = new_mats[x2][y2], new_mats[x1][y
1]

# Setting the no. of misplaced tiles
costs = calculateCosts(new_mats, final)

new_nodes = nodes(parent, new_mats, new_empty_tile_posi,
                  costs, levels)
return new_nodes

# func to print the N by N matrix
def printMatsrix(mats):

    for i in range(n):
        for j in range(n):
            print("%d " % (mats[i][j]), end = " ")

        print()

# func to know if (x, y) is a valid or invalid
# matrix coordinates
def isSafe(x, y):

    return x >= 0 and x < n and y >= 0 and y < n

# Printing the path from the root node to the final node
def printPath(root):

    if root == None:
        return

    printPath(root.parent)
    printMatsrix(root.mats)
    print()

# method for solving N*N - 1 puzzle algo
# by utilizing the Branch and Bound technique. empty_tile_posi is
# the blank tile position initially.
def solve(initial, empty_tile_posi, final):

    # Creating a priority queue for storing the live
    # nodes of the search tree
    pq = priorityQueue()

    # Creating the root node
    costs = calculateCosts(initial, final)
    root = nodes(None, initial,
                empty_tile_posi, costs, 0)

    # Adding root to the list of live nodes
    pq.push(root)

```

```

# Discovering a live node with min. costs,
# and adding its children to the list of live
# nodes and finally deleting it from
# the list.
while not pq.empty():

    # Finding a live node with min. estimatsed
    # costs and deleting it form the list of the
    # live nodes
    minimum = pq.pop()

    # If the min. is ans node
    if minimum.costs == 0:

        # Printing the path from the root to
        # destination;
        printPath(minimum)
        return

    # Generating all feasible children
    for i in range(n):
        new_tile_posi = [
            minimum.empty_tile_posi[0] + rows[i],
            minimum.empty_tile_posi[1] + cols[i], ]

        if isSafe(new_tile_posi[0], new_tile_posi[1]):

            # Creating a child node
            child = newNodes(minimum.mats,
                             minimum.empty_tile_posi,
                             new_tile_posi,
                             minimum.levels + 1,
                             minimum, final,)

            # Adding the child to the list of live nodes
            pq.push(child)

# Main Code

# Initial configuration
# Value 0 is taken here as an empty space
initial = [ [ 1, 2, 3 ],
             [ 5, 6, 0 ],
             [ 7, 8, 4 ] ]

# Final configuration that can be solved
# Value 0 is taken as an empty space
final = [ [ 1, 2, 3 ],
           [ 5, 8, 6 ],
           [ 0, 7, 4 ] ]

# Blank tile coordinates in the
# initial configuration
empty_tile_posi = [ 1, 2 ]

# Method call for solving the puzzle
solve(initial, empty_tile_posi, final)

```

1	2	3
5	6	0
7	8	4

1	2	3
5	0	6
7	8	4

1	2	3
5	8	6
7	0	4

1	2	3
5	8	6
0	7	4

b. Thuật toán tìm kiếm A* trên đồ thị:

*# I found this article very helpful:
<https://www.pythonpool.com/a-star-algorithm-python/>*

```
from collections import deque
```

```
class Graph:
```

```
    def __init__(self, adjac_lis):
        self.adjac_lis = adjac_lis
```

```
    def get_neighbors(self, v):
        return self.adjac_lis[v]
```

This is heuristic function which is having equal values for all nodes

```
    def h(self, n):
```

```
        H = {
            'A': 1,
            'B': 1,
            'C': 1,
            'D': 1
        }
```

```
        return H[n]
```

```
    def a_star_algorithm(self, start, stop):
```

*# In this open_lst is a list of nodes which have been visited, but w
ho's*

*# neighbours haven't all been always inspected, It starts off with t
he start
#node*

```
        # And closed_lst is a list of nodes which have been visited
        # and who's neighbors have been always inspected
        open_lst = set([start])
        closed_lst = set([])
```

*# poo has present distances from start to all other nodes
the default value is +infinity*

```
        poo = {}
        poo[start] = 0
```

par contains an adjac mapping of all nodes

```
        par = {}
        par[start] = start
```

```

while len(open_lst) > 0:
    n = None

    # it will find a node with the lowest value of f() -
    for v in open_lst:
        if n == None or poo[v] + self.h(v) < poo[n] + self.h(n):
            n = v;

    if n == None:
        print('Path does not exist!')
        return None

    # if the current node is the stop
    # then we start again from start
    if n == stop:
        reconst_path = []

        while par[n] != n:
            reconst_path.append(n)
            n = par[n]

        reconst_path.append(start)

        reconst_path.reverse()

        print('Path found: {}'.format(reconst_path))
        return reconst_path

    # for all the neighbors of the current node do
    for (m, weight) in self.get_neighbors(n):
        # if the current node is not present in both open_lst and close
        # add it to open_lst and note n as it's par
        if m not in open_lst and m not in closed_lst:
            open_lst.add(m)
            par[m] = n
            poo[m] = poo[n] + weight

        # otherwise, check if it's quicker to first visit n, then m
        # and if it is, update par data and poo data
        # and if the node was in the closed_lst, move it to open_lst
        else:
            if poo[m] > poo[n] + weight:
                poo[m] = poo[n] + weight
                par[m] = n

            if m in closed_lst:
                closed_lst.remove(m)
                open_lst.add(m)

    # remove n from the open_lst, and add it to closed_lst
    # because all of his neighbors were inspected
    open_lst.remove(n)
    closed_lst.add(n)

    print('Path does not exist!')
    return None

```

Input:



```
adjac_lis = {  
    'A': [('B', 1), ('C', 3), ('D', 7)],  
    'B': [('D', 5)],  
    'C': [('D', 12)]  
}  
graph1 = Graph(adjac_lis)  
graph1.a_star_algorithm('A', 'D')
```



```
Path found: ['A', 'B', 'D']  
['A', 'B', 'D']
```

III. Bài tập ở lớp

Bài 1. Cài đặt thuật giải AKT vào bài toán 15 puzzle($n=4$).

Bài 2. Cài đặt thuật giải A* tìm đường đi giữa 2 đỉnh bất kỳ trong đồ thị.

IV. Bài tập về nhà

Bài 1. Cài đặt thuật giải AKT vào bài toán 15 puzzle($n>4$).

Bài 2. Cài đặt bài toán người giao hàng theo thuật giải A*.

Thuật giải Minimax 2:

```
"""
    TicTacToe Game is a board game of placing crosses and circles and is
    played as a multiplayer or as a computer vs player.
    Copyright (C) 2018 Rahul Gautham Putcha
    This program is free software: you can redistribute it and/or modify
    it under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.
    This program is distributed in the hope that it will be useful,
    but WITHOUT ANY WARRANTY; without even the implied warranty of
    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
    GNU General Public License for more details.
    You should have received a copy of the GNU General Public License
    along with this program. If not, see <http://www.gnu.org/licenses/>.
    For more details on contact, please visit https://rahulgputcha.com or
    email to rahulgautham95@gmail.com
"""
# Header Files
import cv2
import numpy as np
from random import randint
import time

#
-----

# Classes
class Block():
    def __init__(self, i, j):
        self.value = None
        self.pos = (i, j)

    def setValue(self, value):
        self.value = value

#
-----

class GUI():
    def __init__(self, windowName):
        self.windowName = windowName
        self.width, self.height = 400, 400
        self.menuHeight = 100
        self.image = np.zeros((self.height + self.menuHeight, self.width, 3),
np.uint8)
        self.turn = 1
        self.vsCom = 0
        self.reset()

#
-----

# Reset Game
def reset(self):
    self.blocks = []
    self.win = False
    self.change = True
    self.selected = False
```

```

        for i in range(3):
            row = []
            for j in range(3):
                row.append([Block(i, j), (j * (self.width // 3) + 3, i *
(self.height // 3) + 3),
                            ((j + 1) * (self.width // 3) - 3, (i + 1) *
(self.height // 3) - 3)])
            self.blocks.append(row)

-----
#
-----
# Drawing GUI and Game Screen
def draw(self):
    self.image = np.zeros((self.height + self.menuHeight, self.width, 3),
np.uint8)
    for i in range(3):
        for j in range(3):
            start_point = self.blocks[i][j][1]
            end_point = self.blocks[i][j][2]
            cv2.rectangle(self.image, start_point, end_point, (255, 255,
255), -1)
            value = " " if self.blocks[i][j][0].value is None else
self.blocks[i][j][0].value
            cv2.putText(self.image, value, (j * (self.width // 3) + 25,
(i * self.height // 3) + 100),
                        cv2.FONT_HERSHEY_SIMPLEX, 5, (0, 0, 0), 5)
        if self.checkWin():
            string = ("Player " + str(
                self.turn) + " Wins" if self.turn != self.vsCom else
"Computer Wins") if self.turn == 1 else (
                "Player " + str(2) + " Win" if self.turn != self.vsCom else
"Computer Win")
        else:
            if not self.checkDraw():
                string = ("Player " + str(
                    self.turn) + "'s Turn" if self.turn != self.vsCom else
"Computer's Turn") if self.turn == 1 else (
                    "Player " + str(2) + "'s Turn" if self.turn !=
self.vsCom else "Computer's Turn")
            else:
                string = "Match Draw!!"
            cv2.putText(self.image, string, (self.width // 2 - 70, self.height +
30), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                        (255, 255, 255), 1)
            cv2.putText(self.image, "R - Reset", (10, self.height + 60),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
            cv2.putText(self.image, "Esc - Exit", (10, self.height + 80),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
            string = "vs Computer" if self.vsCom == 0 else "vs Human"
            cv2.putText(self.image, "Space - " + string, (self.width // 2 + 10,
self.height + 80), cv2.FONT_HERSHEY_SIMPLEX,
                        0.5, (255, 255, 255), 1)

        if self.selected and not (self.checkWin() or self.checkDraw()):
            self.change = True
            self.selected = False
            self.turn *= -1

-----
#
-----
# Game Play Functions
def mainLoop(self): # Game Loop till Esc(Close) button is pressed

```

```

cv2.namedWindow(self.windowName)
cv2.setMouseCallback(self.windowName, self.mouseCall)
try:
    while True and cv2.getWindowProperty(self.windowName, 1) != -1:
        if self.change:
            self.change = False
            self.draw()

            if self.vsCom == self.turn and not (self.checkWin() or
self.checkDraw()):
                block = self.nextMove()
                block.setValue("x" if self.turn == 1 else "o")

                self.selected = True
                self.change = True

                cv2.imshow(self.windowName, self.image)
                # Keyboard Hits
                key = cv2.waitKey(1)
                if key == 27:
                    break
                elif key == ord("r") or key == ord("R"):
                    self.reset()
                    if key == ord(" ") and not (self.checkWin() or
self.checkDraw()):
                        if self.vsCom:
                            self.vsCom = 0
                        else:
                            self.vsCom = self.turn
                            self.change = True
                cv2.destroyAllWindows()
except:
    print("Window is successfully closed")

def checkWin(self):
    self.win = False
    if (self.blocks[0][0][0].value is not None and
self.blocks[0][0][0].value == self.blocks[0][1][0].value ==
self.blocks[0][2][0].value) or (
        self.blocks[1][0][0].value is not None and
self.blocks[1][0][0].value == self.blocks[1][1][0].value ==
self.blocks[1][2][0].value) or (
        self.blocks[2][0][0].value is not None and
self.blocks[2][0][0].value == self.blocks[2][1][0].value ==
self.blocks[2][2][0].value) or (
        self.blocks[0][0][0].value is not None and
self.blocks[0][0][0].value == self.blocks[1][0][0].value ==
self.blocks[2][0][0].value) or (
        self.blocks[0][1][0].value is not None and
self.blocks[0][1][0].value == self.blocks[1][1][0].value ==
self.blocks[2][1][0].value) or (
        self.blocks[0][2][0].value is not None and
self.blocks[0][2][0].value == self.blocks[1][2][0].value ==
self.blocks[2][2][0].value) or (
        self.blocks[0][0][0].value is not None and
self.blocks[0][0][0].value == self.blocks[1][1][0].value ==
self.blocks[2][2][0].value) or (
        self.blocks[2][0][0].value is not None and
self.blocks[2][0][0].value == self.blocks[0][2][0].value ==
self.blocks[1][1][0].value):
        self.win = True
    return self.win

def checkDraw(self):

```

```

        flag = True
        for i in range(3):
            for j in range(3):
                if self.blocks[i][j][0].value == None:
                    flag = False
            return flag

-----#
-----
# Computers Move Decided Using Minmax Algorithm
def nextMove(self): # Decide NextMove of Computer by this return the
                    # block to selected by the Computer
    flag = 0
    blocks = []
    for i in range(3):
        for j in range(3):
            if self.blocks[i][j][0].value == None:
                blocks.append(self.blocks[i][j][0])
    if not (len(blocks) == sum([len(row) for row in self.blocks]) or
len(blocks) == sum(
        [len(row) for row in self.blocks]) - 1 or len(blocks) == 1):
        scoresList = {}
        for block in blocks:
            if block.value == None:
                if self.computerWins(block):
                    scoresList[block] = 50
                elif self.playerWins(block):
                    scoresList[block] = -50
                elif not self.checkDraw():
                    block.value = ("x" if self.turn == 1 else "o")
                    scoresList[block] = self.min_max(1, self.vsCom)
                    block.value = None
            else:
                scoresList[block] = 0
        # Choosing Either Best Closest Winning Score or Next Closest
        # Losing Score
        bestScore = (
            min(scoresList.values()) if abs(min(scoresList.values())) >
abs(max(scoresList.values())) else max(
            scoresList.values()))
        blocks = []
        for block in scoresList:
            if scoresList[block] == bestScore:
                ##print(block.pos,bestScore)
                blocks.append(block)
        choice = blocks[randint(0, len(blocks) - 1)]
        # print(choice.pos)
        return choice

def min_max(self, depth, player): # MinMax Algorithms Function
    scoresList = []
    for row in self.blocks:
        for block in row:
            if block[0].value == None:
                if self.computerWins(block[0]):
                    return (50 - depth)
                elif self.playerWins(block[0]):
                    return (-50 + depth)
            else:
                block[0].value = ("x" if self.turn == 1 else "o")
                scoresList.append(self.min_max(depth + 1, player *
-1))

                block[0].value = None

```

```

        if scoresList:
            return (min(scoresList) if abs(min(scoresList)) >
abs(max(scoresList)) else max(scoresList))
        return 0

    def computerWins(self, block):
        flag = False
        block.value = ("x" if self.vsCom == 1 else "o")
        if self.checkWin(): flag = True
        self.win = False
        block.value = None
        return flag

    def playerWins(self, block):
        flag = False
        block.value = ("x" if self.vsCom != 1 else "o")
        if self.checkWin(): flag = True
        self.win = False
        block.value = None
        return flag

-----#
-----
    # Mouse Click Functions - (For User Players)
    def mouseCall(self, event, posX, posY, flag, param):
        if event == cv2.EVENT_LBUTTONDOWN and not self.win and self.turn !=
self.vsCom:
            self.setBlockInPos(posX, posY)

    def setBlockInPos(self, x, y):
        for i in range(3):
            for j in range(3):
                if self.blocks[i][j][0].value is None and
self.blocks[i][j][1][0] <= x <= self.blocks[i][j][2][0] and \
self.blocks[i][j][2][1] <= y <=
self.blocks[i][j][2][1]:
                    self.blocks[i][j][0].setValue("x" if self.turn == 1 else
"o")
                    self.change = True
                    self.selected = True
                    break

#
-----
-----

    # Main Program
    game = GUI("TicTacToe")
    game.mainLoop()

```

Thuật giải Cắt tỉa Alpha-beta 2:

<https://github.com/Tsu-HaoLiu/AI-tictactoe>

<https://github.com/RichardRivaldo/Bothicc>